

Session 5 · Deck 5.B6

Worked Example: The RBC Model by Projection

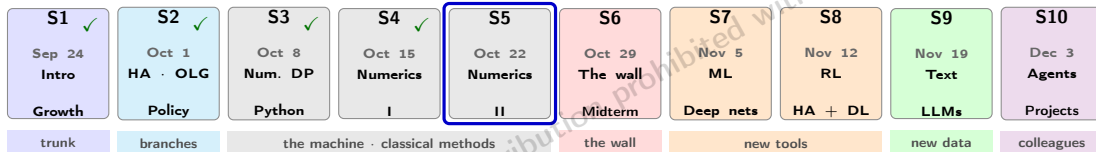
Zhigang Feng

Thursday, October 22, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are

you are here: 5.B6 ▼



Build it on paper → solve it on the machine → push it to the frontier with AI

Block B: 5.B1 the residual · 5.B2 choosing a basis · 5.B3 Chebyshev · 5.B4 finite elements · 5.B5 determining the coefficients · **5.B6** the worked example.

The Same Model, a Fourth Way

Four sessions, one model.

3.A4	grid VFI, everything crude
4.B5	grid VFI, Rouwenhorst + interpolation + golden section + Howard
5.A6	perturbation around k^*
5.B6	Chebyshev collocation on a domain

Here the unknown is the **consumption policy**, written as a Chebyshev polynomial in k for each of the seven shock states, and the coefficients are chosen to zero the Euler residual at the Chebyshev nodes.

The commitments

Basis: Chebyshev in k , degree 7 (5.B3).

Domain: $k \in [0.5k^*, 1.5k^*]$ — chosen, and it matters.

Shock: the same Rouwenhorst 7-state chain as 4.B5, so the comparison is like for like.

Projection: orthogonal collocation (5.B5).

Steps 1 and 2: The Basis, and the Residual

```
def psi(k):                                # [klo, khi] ->
    [-1,1]
    return 2*(k - klo)/(khi - klo) - 1

def cpol(th, k, j):                        # policy for state j
    return chebval(psi(k), th[j])

def resid(th, k, j):
    c = cpol(th, k, j)
    y = z[j]*k**alpha + (1-delta)*k
    kp = np.clip(y - c, klo, khi)
    rhs = 0.0
    for l in range(n_z):
        cp = cpol(th, kp, l)
        R = alpha*z[l]*kp**(alpha-1) + 1-delta
        rhs += Pi[j,l] * R/cp
    return 1.0 - c*beta*rhs
```

Read the last line

That is the **unit-free Euler residual** of 3.A1 — the same expression we have used to grade every solver since Session 3. The difference is that here it is not a diagnostic. It *is* the equation being solved.

The `clip` matters: during the solve the iterate can propose a k' outside the domain, where a Chebyshev polynomial diverges like $\cosh$.

Step 3: Warm Start From Perturbation

```
# 5.A4's four numbers, reused as a starting guess
P, F, _, _ = klein(A, B, n_x=2)

nodes = cos(pi*(2*arange(1,n+2)-1)/(2*(n+1)))
kn = klo + 0.5*(nodes+1)*(khi - klo)

th0 = np.zeros((n_z, n+1))
for j in range(n_z):
    ch = F[0,0]*log(kn/kss) + F[0,1]*logz[j]
    th0[j] = chebfit(psi(kn), css*exp(ch), n)
```

The first-order perturbation policy, evaluated at the Chebyshev nodes and fitted. Six lines, and the nonlinear solve that follows converges from it without trouble.

Ledger worked_proj.warm_start.

This is the honest reason to learn Block

A. Projection needs a good initial guess for a 56-dimensional nonlinear system, and perturbation produces one in under a millisecond.

5.B1 promised the two methods mix rather than compete. This is where it happens, and it is standard practice, not a trick.

Steps 4 and 5: Solve, Then Grade

```
def equations(flat):  
    th = flat.reshape(n_z, n+1)  
    return np.concatenate([resid(th, k_cheb, j)  
                           for j in range(n_z)])  
  
sol = root(equations, th0.ravel(),  
          method="hybr", tol=1e-12)  
th = sol.x.reshape(n_z, n+1)  
# 56 unknowns, 0.13 s, residual 5.1e-15  
  
E = euler_errors(cpol, th, n_test=2000)  
# graded on 2000 RANDOM points, not the nodes
```

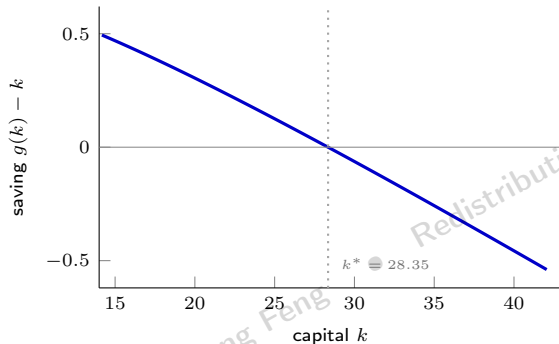
Ledger worked_proj: degree 7, 56 unknowns, 0.129 s.

The grading rule

The residual is zero *at the collocation nodes* by construction, so evaluating there would report 10^{-15} and mean nothing. Grade on **different points** — 3.A1's separation of grids, and the single most common way to fool yourself here.

56 unknowns: 8 coefficients $\times$ 7 shock states.

Does It Work? Plot the Saving, Not the Policy



Ledger fig_proj_policy; the deviation is formed by pgfplots from the ledger's (k, g) pairs.

Saving $g(k) - k$ at the middle shock state.
Plotting $g(k)$ itself would show a straight line at 45° and hide everything.

The curve crosses zero at k^* — saving there is 2.8×10^{-6} — is positive below and negative above, and is smooth throughout. Three behaviour checks on one picture.

Compare 4.B5, where the same crossing landed **ten grid spacings** away from k^* because the policy was quantized. Here there is no grid to quantize it.

What One More Coefficient Buys

Re-solve at a range of degrees, everything else identical:

degree	unknowns	$\max \mathcal{E} $	sec
3	28	$10^{-3.84}$	0.06
5	42	$10^{-5.15}$	0.09
7	56	$10^{-6.40}$	0.13
9	70	$10^{-7.63}$	0.18
11	84	$10^{-8.85}$	0.50

About 1.2 decades for every two degrees, dead straight over five decades of error. That is the geometric convergence 5.B3 promised for a smooth function, arriving on schedule.

Ledger worked_proj.rows; all five converged, graded on the same 2,000 random points.

Compare a grid method, where 3.A1 measured the value error falling like h^2 and the policy error like h . To gain 1.2 decades there you multiply the grid by 16. Here you add two coefficients.

This only holds while the policy is **smooth**. Put a binding constraint in the domain and the same table would flatten out at $O(1/n)$ — 5.B2's kinked function, exactly.

One Model, Four Methods

Method	numbers	sec	$\max \mathcal{E} $ near k^*	$\max \mathcal{E} $ wide
global VFI (4.B5)	2,100	0.94	$10^{-2.82}$	$10^{-2.51}$
perturbation, 1st order	4	< 0.001	$10^{-4.01}$	$10^{-2.56}$
perturbation, 2nd order	12	0.02	$10^{-6.14}$	$10^{-4.50}$
Chebyshev collocation	56	0.13	$10^{-6.82}$	$10^{-6.40}$

The grid method is last on every measure. It stores $37\times$ more numbers than collocation, takes $7\times$ longer, and is **four decades** less accurate.

The flat column

Only collocation is nearly as accurate far from k^* as near it: $10^{-6.40}$ against $10^{-6.82}$. That is what “global” means, measured.

Ledger three_methods. Identical model, chain, domain and residual definition; the 4.B5 row reproduces that deck's own ledger to 0.002 decades and its 66 policy updates exactly.

So Why Did We Spend Two Sessions on Grids?

Because the table is about **this model**: smooth, one-dimensional and unconstrained — the best case for a polynomial basis and the worst for a grid. Change any one and the ranking moves.

- **A binding constraint.** The policy kinks; Chebyshev gets Gibbs, perturbation does not exist, and VFI does not care.
- **A discrete choice.** Same, worse.
- **More dimensions.** The grid dies first, then complete polynomials, then Smolyak.

The decision

Smooth, low- d , want speed → perturbation.

Smooth, need global accuracy → projection.

Kinks, constraints, discrete choice → VFI or EGM.

High- d and smooth → Smolyak, then neural nets (S7).

And whichever you choose, grade it on points you did not solve at.

Comparisons of this kind are often attributed to Aruoba, Fernández-Villaverde & Rubio-Ramírez (2006), which compares VFI, perturbation, Chebyshev projection, finite elements and PEA on this model. The table above is our own run, on our own calibration.

What We Deliberately Did Not Do

- **No constraint.** The one case where the grid methods win, and the case every model in Session 2 has.
- **No second endogenous state.** At $d = 2$ the tensor basis is $8^2 = 64$ coefficients per shock state; at $d = 5$ it is 32,768. That is why 5.B3 spent three frames on Smolyak.
- **We chose the domain by hand.** Too narrow and the solution is wrong where you need it; too wide and the polynomial wastes degrees. Nothing in the method chooses it.
- **No guarantee off the domain.** Outside $[k_{lo}, k_{hi}]$ the approximation does not degrade gracefully — it diverges.

The honest summary

Projection is the most accurate method in this course so far, on the problems it suits, and it has **two free choices** — basis and domain — that it cannot make for you and will not warn you about.

Perturbation had one such choice (the expansion point) and it was forced. Projection has two and neither is.

Takeaways

- The same model, a fourth way: an 8-term Chebyshev polynomial per shock state, coefficients chosen to **zero the Euler residual at the Chebyshev nodes**. 56 unknowns, 0.13 seconds.
- **Warm-start from perturbation**. Block A's four numbers give the nonlinear solver its initial guess — the two methods cooperate rather than compete.
- **Grade on points you did not solve at**. At the collocation nodes the residual is 10^{-15} by construction and means nothing.
- Measured convergence: 1.2 **decades for every two degrees**, straight over five decades. That is the geometric rate smoothness buys.
- **One model, four methods**: the grid solver stores $37\times$ more numbers, runs $7\times$ slower and is **four decades** less accurate than collocation — and only collocation is as accurate far from k^* as near it.
- That ranking is a property of a **smooth, one-dimensional, unconstrained** model. Add a binding constraint and it reverses, which is the single most important sentence in this session.

Bridge: The Models We Actually Care About

Session 5 ends with the best solution of the growth model this course will produce by classical means. It also ends with the condition that made it possible, stated four times: **smooth, one-dimensional, unconstrained**.

Session 2's models are none of those. Aiyagari households sit at a borrowing limit; Krusell-Smith carries a distribution as a state; the life-cycle model has eighty cohorts. Every method in Sessions 3 to 5 has now met the case that defeats it.

Next week we take these tools to those models, and find out exactly where they stop.

Next week (S6)

Heterogeneous agents by classical methods: **Aiyagari** as a nested fixed point, **Krusell-Smith, OLG** and transitions — and the complexity wall that Sessions 7 and 8 exist to break.

Homework 1 was due today. The **practice midterm** is posted this afternoon; the **midterm** is next Thursday, Block B, closed book, covering Sessions 1–5.